DATA GOVERNANCE FOUNDATIONS

Data Security Basics & Audit Readiness

Encryption • Access Control • Masking • Compliance

Hands-on with Databricks & Unity Catalog

Created by Prachi Kabra

Instructor-led | Level: Intermediate | Duration: ~3 hours

Table of Contents

0. About This Module

This module builds the security and compliance foundation every data practitioner needs on a modern lakehouse. It covers the three pillars that protect data — encryption, access control, and masking — and then shows how to stay continuously audit-ready and map controls to real regulations. All labs run on Databricks, using Unity Catalog as the security and audit backbone.

Learning Objectives

- Explain the CIA triad and the shared-responsibility model for cloud data platforms.
- Describe encryption at rest and in transit, and manage keys and secrets in Databricks.
- Design least-privilege access using the Unity Catalog privilege model and groups.
- Apply dynamic column masks and row filters to protect sensitive data on read.
- Use Unity Catalog audit logs, system tables, and lineage as compliance evidence.
- Map platform controls to GDPR, HIPAA, PCI-DSS, SOX, and RBI expectations.
- Run a documentation-and-controls audit and produce an evidence pack.

Prerequisites

- Working knowledge of SQL and basic Python.
- A Databricks workspace with Unity Catalog enabled.
- Ability to create catalogs/schemas/tables, or the equivalent grants from an admin.
- Account-admin or metastore-admin access is needed for the audit-log and system-table sections (an admin can demo these).

> **Environment note**
> Access control, masks, row filters, audit logs, and system tables are Unity Catalog features. They are not available on the legacy hive_metastore. Always use three-level names (catalog.schema.object) and attach notebooks to a UC-enabled cluster or SQL warehouse.

1. Data Security Fundamentals

1.1 The CIA triad

Every security control ultimately serves one of three goals. Naming the goal a control protects keeps design honest.

Pillar	Goal	Example control
Confidentiality	Only authorized parties can read data	Access control, encryption, masking
Integrity	Data is accurate and unaltered	Constraints, checksums, audit trails
Availability	Data is accessible when needed	Backups, replication, DR, uptime SLAs

1.2 Defense in depth

No single control is sufficient. Layer them so a failure in one does not expose the data. On Databricks the layers are: network (private connectivity, IP access lists), identity (SSO, SCIM, MFA), platform (workspace and cluster policies), data (Unity Catalog privileges, masks, row filters), and encryption (at rest and in transit) — all observed by audit logging.

1.3 The shared-responsibility model

In a managed lakehouse, the cloud provider and Databricks secure the infrastructure; you remain responsible for who accesses your data, how it is classified, and how it is governed. Security incidents most often arise from the customer side of the line — over-broad grants, unmasked PII, leaked secrets — which is exactly what this module addresses.

Layer	Owned by provider / Databricks	Owned by you
Physical & host	Yes	No
Storage encryption	Default managed keys	Optional customer-managed keys
Identity & access	Mechanism (UC, SSO)	Actual grants & group design
Data classification	No	Yes — tags, masks, policies

2. Encryption

2.1 Two states to protect

State	Meaning	How it is protected
At rest	Data stored on disk / object storage	AES-256 on cloud storage (S3/ADLS/GCS)
In transit	Data moving over the network	TLS 1.2+ between all components
In use	Data in memory during processing	Isolation, access control, confidential compute

On Databricks, encryption at rest and in transit is on by default: the underlying cloud storage is encrypted with AES-256, and all traffic between the control plane, compute plane, and clients uses TLS. You do not switch encryption on — you decide who manages the keys and how secrets are handled.

2.2 Key management

Encryption is only as strong as its key custody. Two models:

- Platform-managed keys — the default; the provider generates and rotates keys. Zero effort, suitable for most workloads.
- Customer-managed keys (CMK) — you supply and control keys in the cloud KMS (AWS KMS, Azure Key Vault, GCP KMS). Required by many regulated organizations because it lets you revoke access by disabling the key.

> **When to require CMK**
> Choose customer-managed keys when a regulation or contract demands that the data owner be able to render data unreadable independently of the platform. It adds operational responsibility (key rotation, availability), so do not adopt it reflexively.

2.3 Secrets — never hard-code credentials

The most common real-world breach is a credential committed to a notebook or repo. Databricks Secrets store credentials encrypted and inject them at runtime so they never appear in code or logs.

```
# Databricks CLI: create a scope and put a secret
# databricks secrets create-scope prod-scope
# databricks secrets put-secret prod-scope db_password

# In a notebook: retrieve without ever printing the value
password = dbutils.secrets.get(scope="prod-scope", key="db_password")

jdbc_url = "jdbc:postgresql://host:5432/sales"
df = (spark.read.format("jdbc")
      .option("url", jdbc_url)
      .option("user", "svc_reader")
      .option("password", password)     # redacted in logs automatically
      .option("dbtable", "public.orders")
      .load())
```

> **Golden rule**
> If a credential appears in plain text anywhere — a notebook cell, a config file, a Git commit, a print statement — treat it as compromised and rotate it. Databricks automatically redacts secret values from cell output and logs.

3. Access Control

3.1 Principle of least privilege

Grant the minimum access required to do the job, and no more. Access should flow to groups, not individuals, so that joiners and leavers are handled by group membership rather than dozens of object-level edits.

3.2 The Unity Catalog privilege model

Unity Catalog secures objects in a three-level hierarchy: catalog → schema → table/view/function. Privileges are inherited downward, and a consumer needs a traversal privilege at each level above the object.

Privilege	Grants the ability to	Level
USE CATALOG	Traverse into a catalog	Catalog
USE SCHEMA	Traverse into a schema	Schema
SELECT	Read a table or view	Table / view
MODIFY	Insert / update / delete rows	Table
EXECUTE	Run a function	Function
ALL PRIVILEGES	Everything on the object	Any

Grant read access the right way (to a group)

```
-- Least-privilege read for an analyst group on the gold layer
GRANT USE CATALOG ON CATALOG prod            TO `analysts`;
GRANT USE SCHEMA  ON SCHEMA  prod.gold       TO `analysts`;
GRANT SELECT      ON SCHEMA  prod.gold       TO `analysts`;

-- Engineers can write to silver; analysts cannot
GRANT MODIFY ON SCHEMA prod.silver TO `data_engineers`;

-- Inspect who has what
SHOW GRANTS ON TABLE prod.gold.fct_order_line;
```

Revoke and review

```
REVOKE SELECT ON SCHEMA prod.silver FROM `analysts`;

-- Audit all privileges across the metastore (system table)
SELECT grantee, privilege_type, table_catalog, table_schema, table_name
FROM system.information_schema.table_privileges
WHERE table_schema IN ('silver','gold')
ORDER BY grantee;
```

> **Ownership matters**
> The owner of an object can always access and re-grant it, regardless of explicit grants. Set object ownership to a group (not a departing individual) so control never gets stranded.

4. Data Masking

4.1 Why mask instead of just restricting?

Access control decides whether you can query a table at all. Masking and row filtering decide what you see inside it. This lets many users share one governed table while sensitive values are hidden from those who do not need them — no duplicate "safe copies" to maintain.

Technique	What it does	Example
Column mask	Transforms a column value on read	Show only last 4 digits of a card
Row filter	Hides entire rows from some users	Region managers see only their region
Redaction	Replaces value with a constant	email → ***
Tokenization	Swaps value for a reversible token	PAN → tok_9f2c (de-tokenize elsewhere)

4.2 Dynamic column masking in Unity Catalog

A column mask is a UDF attached to a column. It runs at query time and can branch on the caller's group membership, so privileged readers see raw data and everyone else sees a masked version.

```
-- 1) Masking function: full value for pii_readers, else masked
CREATE OR REPLACE FUNCTION prod.security.mask_pan(pan STRING)
RETURN CASE
         WHEN is_account_group_member('pii_readers') THEN pan
         ELSE CONCAT('XXXXXXXX', RIGHT(pan, 4))
       END;

-- 2) Attach to the column
ALTER TABLE prod.silver.dim_customer
  ALTER COLUMN pan_number SET MASK prod.security.mask_pan;

-- 3) Test as a non-privileged user -> sees XXXXXXXX1234
```

4.3 Row-level security (row filters)

A row filter is a UDF returning a boolean; rows for which it returns false are invisible to that user.

```
-- Only show rows for the region the user is entitled to
CREATE OR REPLACE FUNCTION prod.security.region_filter(region STRING)
RETURN is_account_group_member('all_regions')
       OR region = current_user_region();   -- your mapping function

ALTER TABLE prod.gold.fct_sales
  SET ROW FILTER prod.security.region_filter ON (region);
```

> **Combine, don't duplicate**
>
> A single gold table with a column mask on PAN and a row filter on region can safely serve executives, regional managers, and analysts at once. That is the whole point of governed, in-place security — fewer copies means a smaller attack surface and less to audit.

5. Audit Readiness

5.1 What "audit-ready" means

Being audit-ready means you can answer, on demand and with evidence, three questions: Who can access sensitive data? Who actually did? And can you prove your controls were in force the whole time? A last-minute scramble before an audit is a sign the controls are not really operational. The goal is continuous readiness.

5.2 Audit logs and system tables

Unity Catalog records every action — grants, reads, writes, mask changes, logins — in audit logs, surfaced as queryable system tables. This is your primary evidence source.

```
-- Who accessed a sensitive table in the last 7 days?
SELECT event_time, user_identity.email AS user, action_name,
       request_params.full_name_arg    AS object
FROM system.access.audit
WHERE action_name IN ('getTable','generateTemporaryTableCredential')
  AND request_params.full_name_arg = 'prod.silver.dim_customer'
  AND event_time > current_timestamp() - INTERVAL 7 DAYS
ORDER BY event_time DESC;

-- Track privilege changes (grants/revokes) for review
SELECT event_time, user_identity.email AS changed_by, action_name,
       request_params
FROM system.access.audit
WHERE action_name IN ('updatePermissions','grant','revoke')
ORDER BY event_time DESC
LIMIT 100;
```

5.3 Lineage and tags as evidence

Automated lineage proves where regulated data flows, and classification tags prove you identified it. Together they answer an auditor’s "show me everywhere customer PII lives and moves" without manual archaeology.

```
-- Every column tagged as PII across the estate (evidence of classification)
SELECT catalog_name, schema_name, table_name, column_name, tag_value
FROM system.information_schema.column_tags
WHERE tag_name = 'pii'
ORDER BY schema_name, table_name;
```

5.4 Continuous monitoring

Turn the queries above into scheduled Databricks SQL alerts and dashboards so drift is caught automatically. Practical monitors to schedule:

- Over-broad grants — anyone granted ALL PRIVILEGES or access to raw PII.
- Unmasked sensitive columns — PII-tagged columns with no SET MASK.
- Undocumented certified tables — gold tables missing an owner tag.
- Failed access attempts and unusual access patterns.
- Secret scope changes and new service principals.

6. Compliance

6.1 Major regulations at a glance

You rarely need to memorize regulations, but you must recognize what each demands of a data platform so you can map controls to it.

Regulation	Protects	Core platform demand
GDPR (EU)	Personal data of EU residents	Consent, right to erasure, access logs
HIPAA (US)	Health information (PHI)	Encryption, access control, audit trails
PCI-DSS	Payment card data	Masking/tokenization, restricted access
SOX (US)	Financial reporting integrity	Change control, segregation of duties, audit
RBI / DPDP (India)	Financial & personal data	Localization, access control, auditability

6.2 Mapping Databricks controls to requirements

The reassuring news: the techniques in this module satisfy most technical control requirements across regulations. The mapping is largely one-to-many.

Requirement	Databricks control (from this module)
Encrypt sensitive data	Default AES-256 at rest, TLS in transit, optional CMK
Restrict access to authorized users	Unity Catalog grants to groups, least privilege
Hide/deidentify PII	Column masks, row filters, tokenization
Prove who accessed what	system.access.audit logs, dashboards
Demonstrate data mapping	Lineage + PII classification tags
Protect credentials	Databricks Secrets, no hard-coded keys

6.3 Right to erasure (GDPR / DPDP)

"Delete my data" requests require you to locate and remove an individual across the estate. Lineage tells you where the data flowed; Delta gives you the delete. Note that Delta time-travel history and backups must also be handled per your retention policy.

```
-- Locate (via lineage/tags) then delete a subject from the source of truth
DELETE FROM prod.silver.dim_customer
WHERE customer_id = :subject_id;

-- Purge historical versions so the data is truly unrecoverable
VACUUM prod.silver.dim_customer RETAIN 168 HOURS;  -- per retention policy
```

> **Compliance is a program, not a query**
>
> Tools enforce controls; people and process make them defensible. Pair the technical controls here with named data stewards, documented policies, and periodic access reviews. Consult your legal/compliance team for obligations specific to your jurisdiction and industry.

7. Hands-On Lab

Secure a dataset end to end: grant least-privilege access, mask PII, add a row filter, then produce audit evidence. Allow ~45 minutes. A metastore/account admin is needed for the audit-log step.

Step 1 — Set up objects and a security schema

```
CREATE CATALOG IF NOT EXISTS training;
CREATE SCHEMA  IF NOT EXISTS training.silver;
CREATE SCHEMA  IF NOT EXISTS training.security;

CREATE TABLE training.silver.dim_customer (
  customer_id INT, customer_name STRING,
  pan_number STRING, region STRING, email STRING
);
INSERT INTO training.silver.dim_customer VALUES
  (1,'Asha Rao','ABCDE1234F','South','asha@example.com'),
  (2,'Liam Ford','ZXCVB6789K','West','liam@example.com');
```

Step 2 — Grant least-privilege read

```
GRANT USE CATALOG ON CATALOG training       TO `analysts`;
GRANT USE SCHEMA  ON SCHEMA  training.silver TO `analysts`;
GRANT SELECT      ON TABLE   training.silver.dim_customer TO `analysts`;
```

Step 3 — Mask PAN and email

```
CREATE OR REPLACE FUNCTION training.security.mask_pan(p STRING)
RETURN CASE WHEN is_account_group_member('pii_readers') THEN p
            ELSE CONCAT('XXXXXX', RIGHT(p,4)) END;

ALTER TABLE training.silver.dim_customer
  ALTER COLUMN pan_number SET MASK training.security.mask_pan;
```

Step 4 — Add a row filter by region

```
CREATE OR REPLACE FUNCTION training.security.region_filter(r STRING)
RETURN is_account_group_member('all_regions') OR r = 'South';

ALTER TABLE training.silver.dim_customer
  SET ROW FILTER training.security.region_filter ON (region);
```

Step 5 — Produce audit evidence

```
-- Confirm the grants you made
SHOW GRANTS ON TABLE training.silver.dim_customer;

-- (Admin) recent access to the table
SELECT event_time, user_identity.email, action_name
FROM system.access.audit
WHERE request_params.full_name_arg = 'training.silver.dim_customer'
ORDER BY event_time DESC LIMIT 20;
```

Deliverable

Submit: (1) the SHOW GRANTS output, (2) a screenshot showing masked PAN when queried as a non-privileged user, and (3) one sentence on which regulation each control (mask, filter, grant) helps satisfy.

8. Knowledge Check

Check understanding at the end of the session. Answers follow.

1. Name the three pillars of the CIA triad and give one control that serves each.
2. What is the difference between encryption at rest and in transit, and is either off by default on Databricks?
3. Why should access be granted to groups rather than individuals?
4. A user needs to read prod.gold.fct_sales. List the three grants required and why.
5. When would you choose customer-managed keys over platform-managed keys?
6. Explain the difference between a column mask and a row filter, with an example of each.
7. Which system table proves who accessed a table, and what is one column you would select from it?
8. How do lineage and PII tags together help satisfy a GDPR "data mapping" request?
9. Give one Databricks control that helps satisfy each of: PCI-DSS, HIPAA, and SOX.

Answer Key

1. Confidentiality (access control/encryption/masking), Integrity (constraints/audit trails), Availability (backups/replication/DR).

2. At rest protects stored data (AES-256 on cloud storage); in transit protects moving data (TLS). Both are on by default on Databricks.

3. Group-based grants make joiners/leavers a single membership change and prevent access being stranded on a departing individual; they are auditable and consistent.

4. GRANT USE CATALOG ON the catalog, GRANT USE SCHEMA ON the schema, and GRANT SELECT ON the table — because UC requires a traversal privilege at every level above the object plus read on the object.

5. When a regulation or contract requires the data owner to be able to render data unreadable independently of the platform (e.g., by disabling the key), accepting the added key-management responsibility.

6. A column mask transforms a value on read (e.g., show only last 4 digits of a PAN); a row filter hides whole rows from some users (e.g., a manager sees only their region).

7. system.access.audit; useful columns include event_time, user_identity.email, action_name, and request_params.full_name_arg.

8. Lineage shows everywhere the data flows and PII tags prove which columns are personal data, so together they map all locations of an individual's data without manual tracing.

9. PCI-DSS: masking/tokenization of card data. HIPAA: encryption + access control + audit trails. SOX: change control and audit logs (segregation of duties via grants).

9. Security & Audit Checklist

A one-page reference for learners.

- Confirm encryption at rest and in transit is active; decide platform vs customer-managed keys deliberately.
- Never hard-code credentials — use Databricks Secrets and rotate anything exposed.
- Grant least privilege, always to groups, and set object ownership to a group.
- Classify sensitive data with tags before securing it.
- Mask PII columns and apply row filters instead of making duplicate "safe" copies.
- Review grants regularly with SHOW GRANTS and system.information_schema privileges.
- Schedule audit queries as alerts: over-broad grants, unmasked PII, failed access.
- Keep lineage and tags current — they are your compliance evidence.
- Map every control to the regulation it satisfies; keep an evidence pack ready.
- Handle erasure requests through the source table plus VACUUM per retention policy.

Key takeaway

Encryption, access control, and masking are the three locks; audit logs and lineage are the proof the locks were on. In Databricks, Unity Catalog provides both the locks and the proof — which is what turns "we think we are secure" into "we can demonstrate it".

— End of Module —

Created by Prachi Kabra • Data Governance Foundations